DevOps Report - Group C

Anders Juul Ingerslev (aing@itu.dk)
Asger Ramlau Jørgensen (asjo@itu.dk)
Johan Kristoffer Skoven Løye (jklo@itu.dk)
Malte Black Nielsen (mbln@itu.dk)

18th of May, 2026

Contents

1 System's Perspective

1.1 Design & Architecture

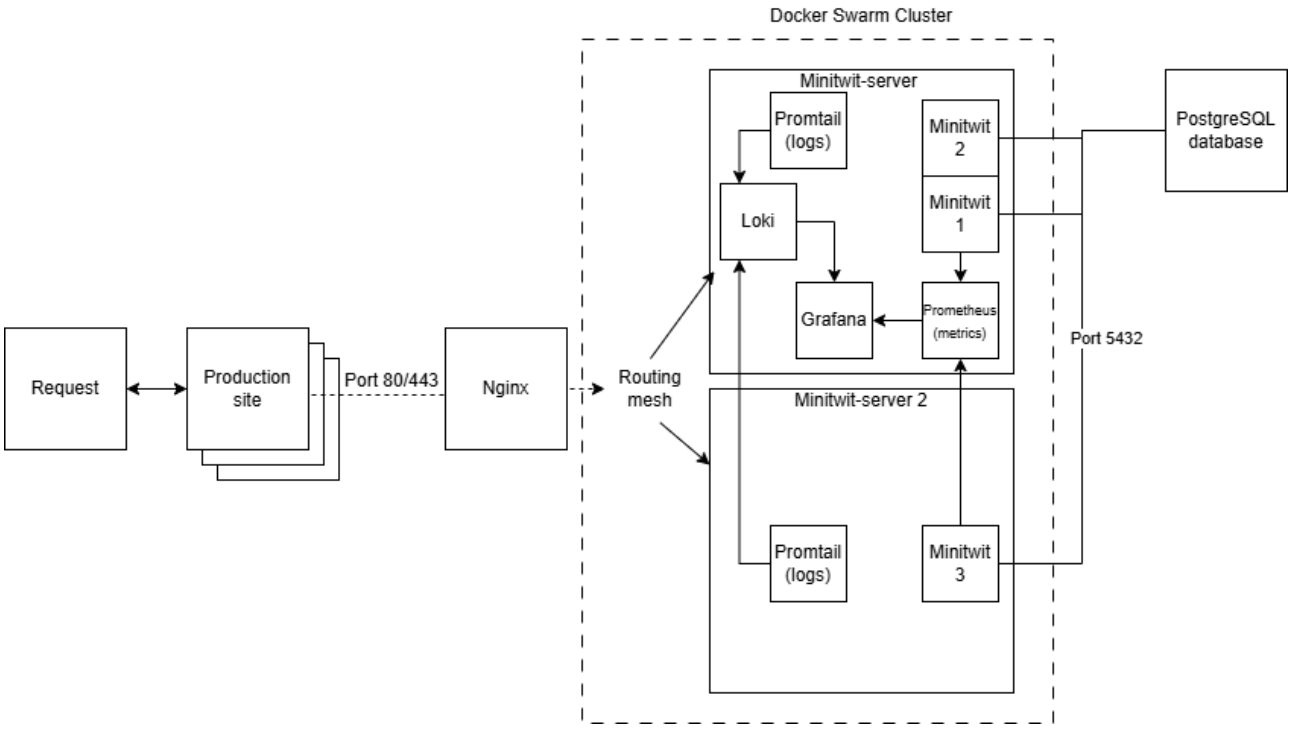

Figure 1: Architecture of the MiniTwit System

1.1.1 Application

The Minitwit web-application is a twitter-like clone with simple functionality where users of the application can post tweets and follow each other. The application is written in Python with Jinja templates. The application has an API that contains the same functionality as the web-app.

1.1.2 Infrastructre

Our system runs on three droplets hosted on DigitalOcean. Two swarm nodes form our swarm cluster. `Minitwit-server` acts as a manager-worker and `Minitwit-server 2` is solely a worker. The last droplet is dedicated to hosting the sytem's Postgress database.

Besides managing container orchestration `Minitwit-server` is hosting two application replicas, a Promtail, Loki and Grafana replica. `Minitwit-server 2` is hosting one application replica and a Promtail replica. Niginx is used as a reversed-proxy listening for incoming requests at `port:80/443`. Nginx forwards the given request and then the routing mesh of Docker Swarm forwards the request to an available replica. The routing mesh of the swarm ensures that each node in the cluster can handle any request regardless of which node is hosting the appropriate container.

The swarm nodes share a `minitwit_minitwit-network` for inter-service network communication between containers on different nodes. The containers hosted on `Minitwit-server` are part of a shared network, `minitwit-network`, which is used for local communication.

1.1.3 Monitoring

Our system monitors the web application using Prometheus, Loki, Promtail and Grafana. Docker has been used for containerization of the system's services.

1.2 Dependencies

1.2.1 Infrastructure

- **DigitalOcean Droplets**: Provides the hardware (CPU, Memory, Storage) to run a Linux OS on, hosting our services.
- **Docker**: Allows us to containerize our services.
- **Docker Swarm**: A part of Docker, allowing us to handle horizontal scaling and manage container health, update strategies and general management.
- **PostgreSQL**: A relational database management system.

1.2.2 Application

- **Fastapi**: An asynchronous web framework handling routing, requests, and endpoints.
- **uvicorn**: A web server deployment dependency used to run the FastAPI application.
- **Jinja2**: A template engine used for server-side rendering of user timelines and layouts.
- **Sqlmodel / SQLAlchemy**: An ORM used to abstract database interactions into Python code.

- **Psycopg2-binary**: A PostgreSQL adapter required to handle high-concurrency network communication with our database container.
- **Werkzeug / itsdangerous** : A Cryptographic utility relied upon for password hashing and session tokens.

1.2.3 Monitoring

- **Prometheus**: A monitoring toolkit used to scrape and monitor time-series metrics.
- **Loki:** An aggregation system to store the logs.
- **Promtail**: A service attached to all containers, which streams container stdout logs to Loki.
- **Grafana**: A data visualization and analytics web application, allowing us to query and monitor the output of the above and organize it in dashboards.

1.2.4 Pipeline

- **GitHub Actions**: Our CI/CD platform that automates our deployment, testing and building.
- **Docker Buildx and QEMU**: Compilation tools allowing us to create Docker Images that are agnostic to the OS they're being run on.
- **Black**: A code formatting tool allowing us to ensure an identical code style.
- **Hadolint**: An analysis tool that scans Docker Images for known vulnerabilities.
- **ShellCheck**: An analysis tool that scans shell files for bugs, syntax errors and vulnerabilities.
- **GitHub CodeQL:** An analysis tool that scans the code base to catch security vulnerabilities (such as SQL injection points or insecure data paths) that standard unit tests overlook.
- **Docker Scout:** A tool for for scanning Docker images for vulnerabilities.
- **Trivy**: A Vulnerability scanner used to scan Docker images.
- **pytest**: A testing library for python.
- **Selenium**: A testing framework for UI tests.

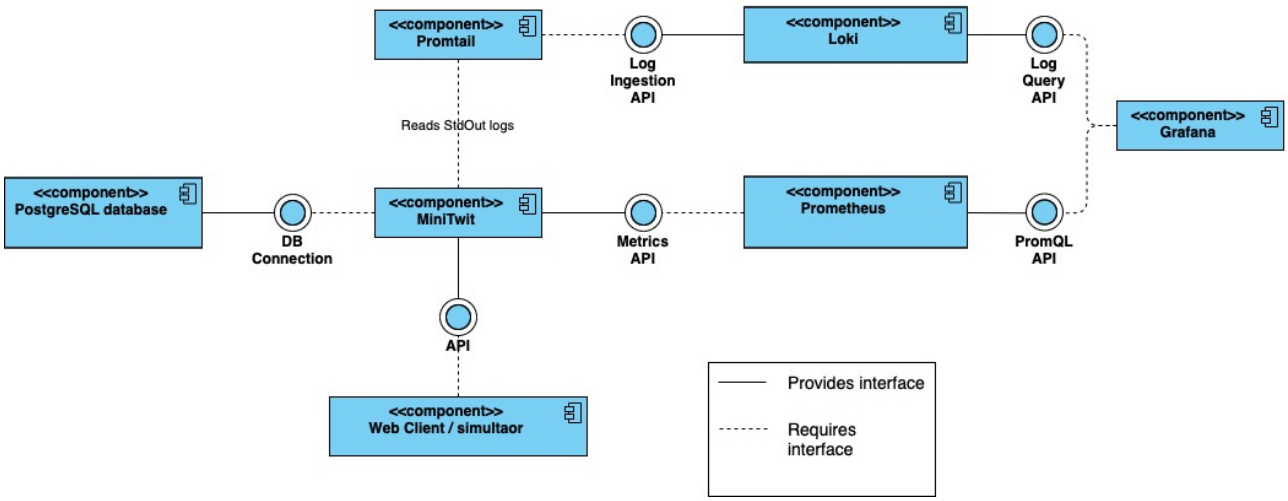

Figure 2: Component Diagram of the Minitwit system, showing provided interfaces and dependencies between the core application, database and monitoring services

1.3 Current System State

Our application is still running on a Docker Swarm cluster across DigitalOcean droplets, with traffic forwarded via Nginx. The data remains persistent on a separate, dedicated droplet running a PostgreSQL database.

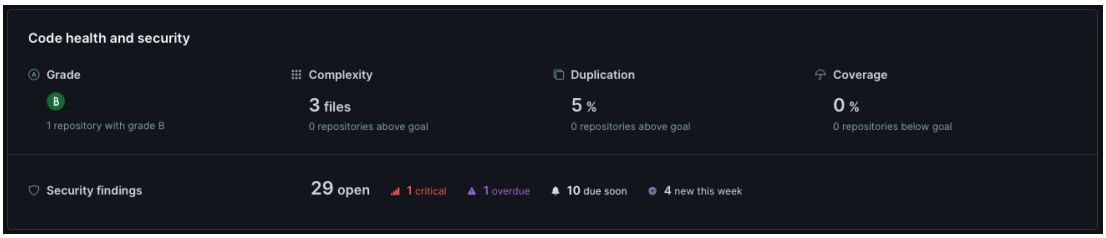

Figure 3: Codacy report on repository

The repository currently has a Grade B for overall code health, with low code duplication (5%) and low file complexity. However, security technical debt is an issue, with several open findings. While this is definitely a pressing issue, we unfortunately did not have the time to remedy the issues. Additionally, while a large suite of automated tests exists within the project, test coverage is reported at 0% due to a configuration issue where the CI/CD pipeline is not yet exporting and uploading test reports to Codacy.
Our production infrastructure is created by our (CI/CD) pipeline, which handles

deployment directly to existing cloud droplets. To ensure reproducibility, we have implemented Infrastructure as Code (IaC) using Terraform. While the production site remains stable on DigitalOcean, the Terraform configurations allows others to automatically provision, configure, and tear down an identical, self-contained environment with an isolated database, and a Docker Swarm cluster. The directions to do this is outlined in the ReadMe file. One large issue with our terraform script though, is that it currently leaves the created DigitalOcean droplets open without a firewall. While we attempted to automate firewall rules, we were not able to do so, as the swarm workers would not be able to connect to the manager. Given more time, this should naturally be remedied.

2 Process' Perspective

2.1 Pipelines

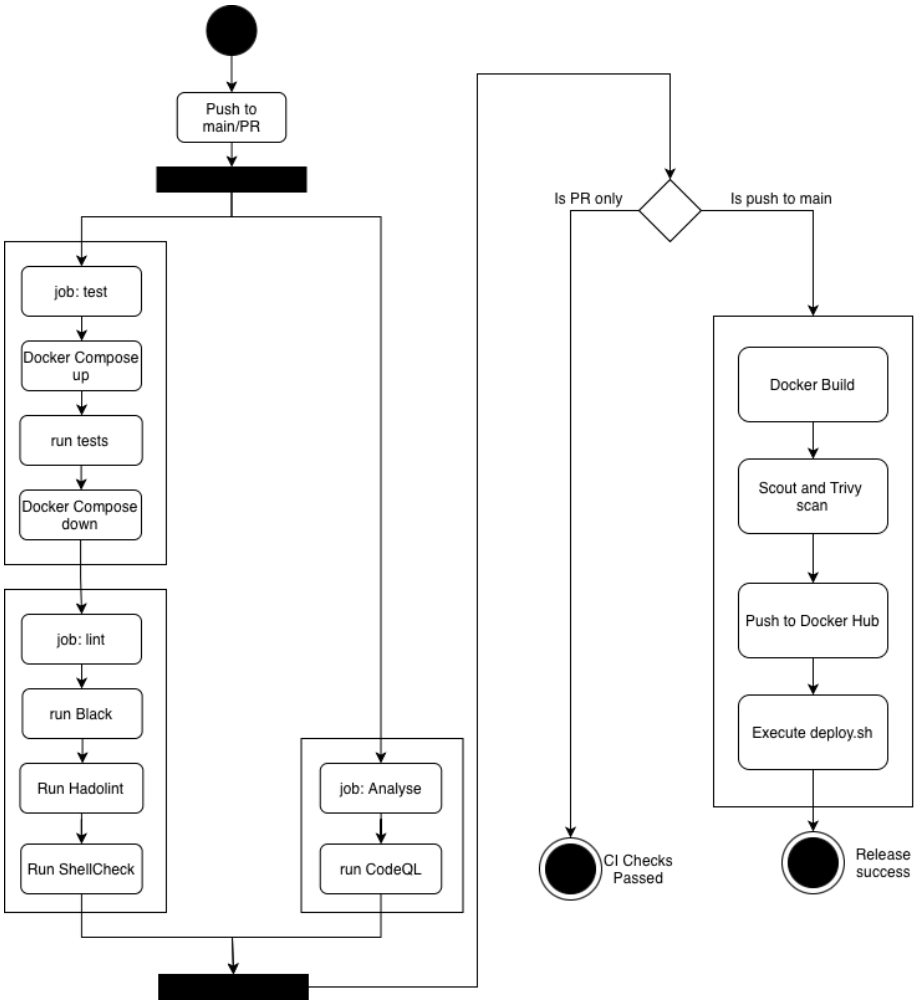

Figure 4: UML Activity Diagram of CI/CD

Our pipeline is automated using GitHub Actions, triggered on every push and pull request to the main branch. We defined a four-stage process for it: test, analyze, lint, build before it deploys. The flow of the pipeline can be seen in Figure 4

2.1.1 Test stage

In the first stage we use a docker compose file to spin up the application in a separate environment with the necessary dependencies. A test database will be created and used in this test environment. The test container then runs all our defined tests via *pytest* against the test database. Once the tests have finished the test database is disposed. If a test fails, the container exits with a non-zero code which stops the pipeline from proceeding further. Having a containerized environment and database ensured that our test suite is isolated and no side effects on our system is caused from the test.

2.1.2 Lint & Analyze

These stages runs the static analysis tools described in 1.2.4 in parallel (See 4) directly against the source code without executing containers or stating the application. The steps reads the source code and files to ensure security and quality standards. If no critical vulnerabilities are found in the steps the pipeline proceeds to the next stage.

2.1.3 Build

This stage is responsible for building the Docker Images, and checking vulnerabilities before they are released to Docker Hub. The image is build with Docker Buildx and QEMU. QEMU allows us to make the resulting image agnostic to the OS on which it is running. This came as a result of the team having a mix of OS architectures, namely ARM (Mac) and x86 (Windows).

Once built, we make use of Docker Scout to scan the image for vulnerabilities. Docker scout is configured with a `fail-on: critical` policy, ensuring that if the image contains a vulnerability marked as "critical" the pipeline is aborted and the image is not pushed. If no vulnerability is found the image is pushed to Docker Hub with credentials safely secured in GitHub Secrets. As an extra layer of security Trivy runs a final security scan.

2.1.4 Release and deployment

Once the image is pushed to Docker Hub, the pipeline starts a Secure Copy Protocol (SCP) to synchronize the folder `remote_files` onto the production server. The pipeline then uses SSH to execute the script `deploy.sh` which manages three tasks:

- 1: Pull the latest image from Docker Hub.
- 2: Run `docker stack deploy`, creating a Docker Swarm to start a cluster of containers.
- 3: Run `docker image prune -f` to remove old images.

The Docker Swarm manages 3 application replicas across the two swarm nodes, ensuring that up to 2 replicas can go down at a time without causing an error for the end-user. The swarm is configured with `order: start-first` which minimizes any downtime and ensures rolling updates. The swarm will spin up a new container with the new image, once healthy it prunes the old container before continuing on to the next container. It is configured with `parallelism: 1` to ensure only one of the three containers in the swarm is updated at a time.

2.2 Monitoring & Logging

2.2.1 Monitoring

Our Prometheus instance is set up to scrape time series metrics every five seconds from all three replicas of the application, and our Grafana replica is used to visualize these metrics on dashboards. The flow can be seen in figure 5.

We maintain the following three dashboards:

- Health dashboard: Tells us the status (UP/DOWN) of the application and Prometheus itself.
- Performance dashboard: Visualizes various metrics of the application to give an indication of the performance. These include; total HTTP requests and rates, total HTTP failures and success rates, response time, etc.
- Database dashboard, showing current amount of unique users, and latest entries.

2.2.2 Logging

A Promtail replica is running on each of our Docker Swarm nodes scraping for logs printed by our containers. The logs are then aggregated by Loki, and Grafana is used for vizualizing the logs. Grafana queries for logs that contain error messages from our application containers.

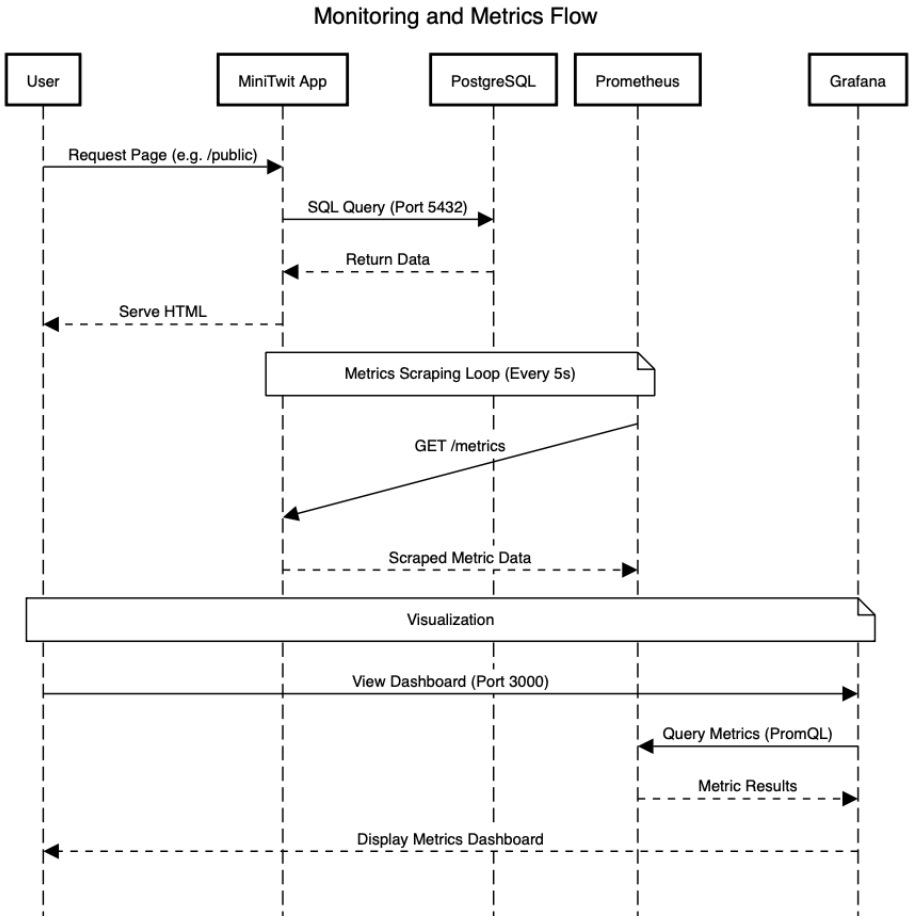

Figure 5: UML Sequence Diagram showing Monitoring flow

2.3 Security

We have implemented Nginx as a reverse proxy, which listens for traffic on port 80 (HTTP) & port 443 (HTTPS). HTTP requests are redirected to HTTPS. Any incoming traffic is forwarded to our Minitwit application, which is running locally on the droplet. Nginx then fetches the response and sends it back to the client. This assures we only have a single point of access.
Certbot manages SSL certificates, ensuring all communication for end users is TLS encrypted.

We use Uncomplicated Firewall (UFW) to only expose certain ports, as can be seen in Figure 9.

The Docker containers running our application are running as a non-privilege user. In this way if a running container is compromised, it will not have any privileges to access or modify the underlying system.

2.4 Availability & Scaling

The application is available at `https://www.helgeandfriends.com/`. We used One.com to register the domain and add an A record to point to our public IP address (161.35.68.148).

As our traffic grew, we needed to scale our setup to accommodate for this increase. We did so by adjusting our Docker setup to swarm mode. Docker Swarm allows for multiple, concurrent services, meaning our application could handle more requests. Additionally, it also helped our availability, as with multiple services, the application would still stay up and running if one were to fail.

3 Reflection Perspective

3.1 Evolution & Refactoring

Refactoring the system to utilize the Object-relational-mapper(ORM) was challenging as it involved a large refactoring of tests and endpoints for our system. Additionally, it required a new way to handle the connection string for the database. It was implemented simultaneously with a migration to a new database, which was also a bit of a challenge as SQLite and Postgress databases' connection strings is treated differently by the ORM.

Furthermore, extra effort had to be made to ensure that the flash messages of the Jinja templates could be used after this refactor, as seen from this commit **3c391f4762837f35d4fcc3b2b84d48812cac43d1**.

3.2 Operation & Maintenance

3.2.1 Deployment of Monitoring Stack

When attempting to deploy our monitoring stack we had issues with building and pushing our images onto Docker Hub through our pipeline. We saw issues where the pipeline reported success, but the server had not pulled the latest deployed images.

This happened because our configuration was trying to build the service locally on the server from source code. As shown in commit `b20ee33b0e54661881a8e1e610f1260f77dea086`, we fixed this by getting the configuration to pull the pre-built image from Docker Hub instead.

Before:

```
prometheus:
build:
context: ../monitoring/prometheus
```

After:

```
prometheus:
image: ${DOCKER_USERNAME}/prometheus_image:latest
```

Evidence from running `docker ps` on the server confirmed this issue. As shown in Appendix 6, the container was still running an old image ID from two days ago, proving that the pipelines success status was misleading because the server had not actually pulled the new updates.

3.2.2 Monitoring with Docker Swarms

After migrating to Docker Swarm our monitoring stack failed to scrape data and showed empty dashboards. (Appendix 7).

Initially we thought to fix this by hard-coding the server's IP address as a target for the data scraping, but we realized that in a swarm environment, containers are dynamic. They start on different nodes with different IP's on every re-start. A hardcoded target would only monitor a single instance, and would break if the Swarm moved the container to a different address.

We resolved this in `Commit 5b452bd9cd28dd3b8b6a78b44c4141168097d986` by switching to DNS service discovery. Instead of a fixed IP, we used the `tasks.` prefix, which tells Prometheus to query Docker's internal DNS for the IP's of all three running replicas `monitoring/prometheus/prometheus.yml`:

```
dns_sd_configs:

names: ['tasks.minitwit_minitwit']
type: 'A'
port: 5001
```

This allowed Prometheus to automatically detect and scrape all three containers, as confirmed by the three active endpoints in Figure 8.

3.2.3 API for simulator

When we initially set up the API for the simulator, we ran into an issue with correctly defining the endpoints. None of us had before used FastAPI, so we were not completely aware of the ordering of the defined endpoints. This lead to about a week of missed data from the simulator. This was fixed in `commit 20898c14851290e95260c65ecea6592bc5829c23`. Because of this, we saw a lot of errors for a short period of time, where the simulator attempted to interact with Minitwit via users that were not stored in our database. We discussed seeing if we could obtain a snapshot of another teams database, but decided that it was unnecessary as new users would, over time, be created by the simulator.

3.3 Workflow

Initially, we struggled to maintain an overview of weekly tasks and distribute them among the team. This led to instances where multiple members unknowingly worked on the same task, resulting in duplicated work.

To help with this we adopted GitHub issues. This workflow gave us a clear overview of our progress and allowed us to link our planning directly to working on the task by creating branches and Pull Requests directly from specific issues.

Our group practiced an informal approach to pair programming, where we collaborated on tasks without using structured roles like navigator and driver, or systematic switching between them. This lack of structure meant we did not mark our commits as co-authored. Similarly, we used LLMs to help us with code generation, debugging, and general sparring, but this use of AI was also not formally documented or credited in our commit history.

We did not maintain a continuous notebook or update our changelog during the project. Furthermore, we did not track the duration of tasks using `/Timespent` on the GitHub issues. Instead, we relied mainly on coordination and communication in person and online. This was not ideal as it made our work history less transparent.

4 Use of Generative AI

For the project we have been using AI (primarily Gemini, ChatGPT and co-pilot) various task which include:

- **Explanation of Concepts:** Clarifying concepts introduced in the lectures as well as trying to understand various project tasks.
- **Debugging:** Analyzing error messages, pipeline failures, and container related issues.
- **Code Generation:** Used to generate boilerplate/template code for configuration files.
- **Report Polish:** Refining section of grammar and flow.

These tools generally increased the speed our workflow and prevented us from spending too much time stuck on minor errors. It has been a good tool for generating templates/boilerplate code especially for some of the technologies that the team was not familiar with. However, as the complexity and size of our system grew, the AI would often misguide the team leading to time wasted. Since this course constantly introduced new concepts and technologies, AI supported our learning process by helping us quickly understand and get started with these new concepts.

5 Appendix

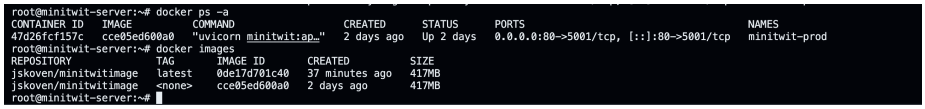

Figure 6: Output of `docker ps -a` Command

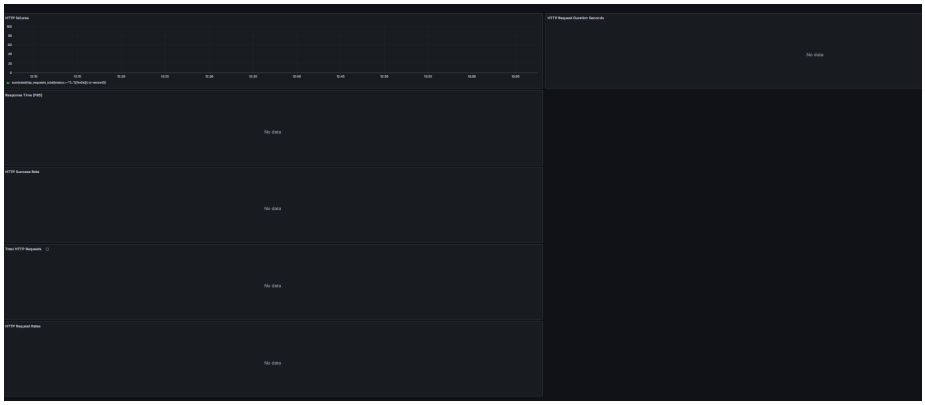

Figure 7: Grafana dashboards showing *"No Data"*

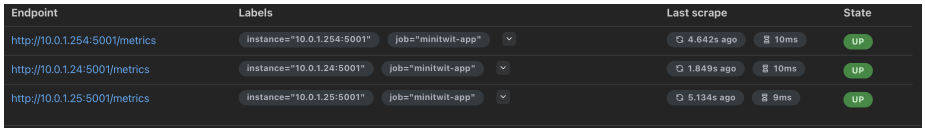

Figure 8: Prometheus health board

```
Status: active

To                         Action      From
--                         ------      ----
22                         ALLOW       Anywhere
80                         ALLOW       Anywhere
443                        ALLOW       Anywhere
22/tcp                     ALLOW       Anywhere
Nginx Full                 ALLOW       Anywhere
```

Figure 9: Firewall enabled ports